Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 1

з дисципліни «Системне програмування вбудованих систем»

на тему «LightBench: середовище STM32 HAL і запуск стенда світла та положення»

Виконав:
студент групи ІО-32
Пашко Максим Ігорович

Перевірив:
Гордієнко Нікіта Юрійович

Київ – 2026

Мета роботи:

Налаштувати середовище Ubuntu 24.04, Docker, STM32 HAL, Wokwi для стенда LightBench. Отримати відтворювану збірку та основу UART/LED, визначити функціональні вимоги до збору світла й положення носія.

Постановка завдання:

1. Зібрати прошивку Cortex-M0+ для Nucleo-C031C6, перевірити два чисті складання.
2. Описати три компоненти з різними інтерфейсами: LDR через ADC, MPU6050 через I²C та кнопку GPIO.
3. Визначити правила часових міток, придатності пари та контрольної точки.
4. Перевірити UART 115200 8N1 та індикацію активної серії у спільній прошивці.

Призначення LightBench:

Оператор порівнює світло та положення носія у вибраних точках. Один LDR дає некалібрований raw, акселерометр дає оцінку нахилу у спокої, кнопка позначає пару. Результатом є записи й статистики серії. У Wokwi світло й прискорення задаються незалежно, тому дослід не доводить фізичної залежності між ними.

Цей звіт описує середовище й вимоги поточної версії LightBench. Драйвери, перевірки часової моделі та журнал RAM розглянуто у ЛР2. Постійне збереження після reset залишається ЛР3.

Функціональні вимоги:

Повний PRD має 14 вимог у `docs/PRD.md`. Вимоги FR-01–FR-14 нижче мають критерії перевірки; FR-12 належить майбутній ЛР3.

- **FR-01** Збирати HAL-прошивку в Docker, друкувати версію й окремі стани ADC, IMU та кнопки через UART 115200 8N1.
- **FR-02** Збирати ADC та IMU кожні 20 мс, зберігати час завершення кожного джерела; інтервал понад 40 мс враховувати як gap.
- **FR-03** Передавати raw 0..4095 без непідтверджених люксів.
- **FR-04** Визначати нахил $\theta = \arccos(a_z/|a|)$ лише за $0{,}85 \leq |a| \leq 1{,}15$ g. Інакше кут invalid.
- **FR-05** Команди run/stop керують серією; нова серія скидає статистики й збільшує номер, але лишає попередні точки.
- **FR-06** Стабільне натискання 30 мс створює одну точку в наступні 50 мс; утримання не повторює запис.
- **FR-07** Точка містить час позначки й обох джерел, raw, кут і причини непридатності. Межі віку 40 мс та розриву 20 мс включні.
- **FR-08** Після stop видавати кількості, failures/gaps, min/max/mean raw та середній придатний кут з окремим знаменником. Порожнеча не означає нульовий кут.
- **FR-09** Зберігати 64 останні точки у RAM, рахувати перезапис найстарішої.
- **FR-10** Експортувати сталий знімок із count і CRC32, продовжуючи збір.
- **FR-11** За відмови IMU продовжувати ADC та пробувати IMU раз на секунду. Відмова ADC завершує серію.
- **FR-12** У ЛР3 відновлювати підтверджений flash-журнал після reset, відхиляти обірвані записи за CRC. У ЛР2 ця функція не реалізована.
- **FR-13** Обробляти run, stop, status, export, selftest та діагностичну зміну адреси; відхиляти невідомі й довгі рядки. Ціль status до 100 мс без черги TX.
- **FR-14** Показувати активну серію LED; розділяти помилки каналів та переповнення UART. Сховище ЛР2 позначати `ram_volatile`.

План перевірки:

Тести LB-T01–LB-T13 у `docs/test-plan.md` пов'язують FR із входами й очікуваними результатами. Окремо перевіряються межі часу, статичні кути 0/90/180°, помилки шини та контрольна точка під час експорту. Host-тест не замінює цифровий захват із симулятора.

Хід роботи:

Середовище та структура:

Обрано форму B, STM32 HAL і модель Nucleo-C031C6. STM32C031C6 має Cortex-M0+, 48 МГц, 32 КіБ flash та 12 КіБ RAM. Команди Make запускають ARM GCC 13.2.1 у Docker на Ubuntu 24.04. Digest базового образу й версії пакетів зафіксовано у Dockerfile. На іншій архітектурі хоста повторюваність треба перевірити окремо.

Core містить початкову конфігурацію MCU, Drivers містить HAL/CMSIS. Devices реалізує MPU6050 через інтерфейс шини. Platform поєднує апаратні операції з runtime; App містить часові потоки, геометрію, серії, точки й протокол команд. Залежності vendor збережено з дозволеної основи SP-Boilerplate та stm32-hello-wokwi з їхніми ліцензіями.

Вузол	Конфігурація
Clock	HSI48; SYSCLK/HCLK/PCLK 48 МГц
USART2	PA2 TX, PA3 RX, AF1, 115200 8N1
I²C1	PB8 SCL, PB9 SDA, AF6, standard mode 100 кГц
ADC1	PA0 IN0, 12 біт, PCLK/2, analog input
Кнопка / LED	PA1 input pulldown, HIGH=pressed; PA5 output
Маркери	PA4 ADC, PB0 точка, PB1 IMU

Файл `firmware.ioc` зберігає базову конфігурацію Core. Додаткові ADC/GPIO та UART IRQ задає `Platform/board.c`; це ручна частина, яка не відтворюється лише повторною генерацією `.ioc`. Монтажним джерелом є `diagram.json`. PA13/PA14 збережено для SWD.

Вибір компонентів:

LDR використовує ADC і дає raw 0..4095. BH1750 не обрано, бо потрібний саме аналоговий канал; потенціометр не реагує на світло. MPU6050 використовує I²C і дає числовий вектор прискорення. Контакт нахилу не замінює його геометрію. Кнопка GPIO дозволяє оператору вибрати момент точки. Автозапис цього рішення не замінює.

Усі вузли працюють зі спільним GND і логікою 3.3 В. MPU AD0 з'єднаний із GND, адреса 0x68. LDR DO, MPU INT/XDA/XCL не використовуються. UART та LED забезпечують відповідь і індикацію без додаткового дисплея. Камера для форми B не потрібна.

Дві чисті збірки:

У корені проєкту двічі виконано:

```
make clean
make build
```

Кожен запуск видалив старі об'єкти та створив ELF, HEX і BIN. Обидва журнали й SHA256 збережено у `docs/evidence`. Компіляція завершилася без попереджень. Результат компонування:

```
RAM:    8456 B / 12 KB  (68.82%)
FLASH: 23936 B / 28 KB  (83.48%)
text=23840 data=96 bss=8368
```

Останні 4 КіБ flash зарезервовані для майбутнього сховища ЛР3. RAM у map включає резерви heap 512 і stack 1024 байти; це не вимір піка стеку. SHA256 обох HEX однаковий:

`a9acd961410a530806ee721eb9e71ba3a87d73e7453ad3dadfd4ef4b9534cbaf`

Перевірений build ID `ed8ba61-d5b4dbed` містить parent commit та checksum прикладних C-файлів. На час запуску виправлення ще не були закомічені, тому точний склад додатково фіксує manifest із хешами вихідних файлів. Дата фактичного тесту 27.10.2026 не виводиться з дат Git.

UART і LED у спільній прошивці:

У VS Code виконано `Wokwi: Start Simulator`. Симулятор завантажує `firmware/build/firmware.hex` і відповідний ELF. TX PA2 підключений до RX монітора, PA3 до TX. Після запуску прочитано:

```
project=LightBench build=ed8ba61-d5b4dbed
adc=1 imu=1 button=ready store=ram_volatile
selftest adc=1 imu=1 button=ready bus=0 ms=178
reply=run result=ok ms=180
```

Перший банер у сирому записі є одним рядком, тут його перенесено для читання. `run` вмикає LED, `stop` вимикає. Для поточної інтегрованої версії індикація означає активну серію, а не демонстраційний blink із затримкою. Періодичний `status` підтверджує продовження роботи; `HAL_Delay` в основному циклі відсутній.

Власний запуск моделі:

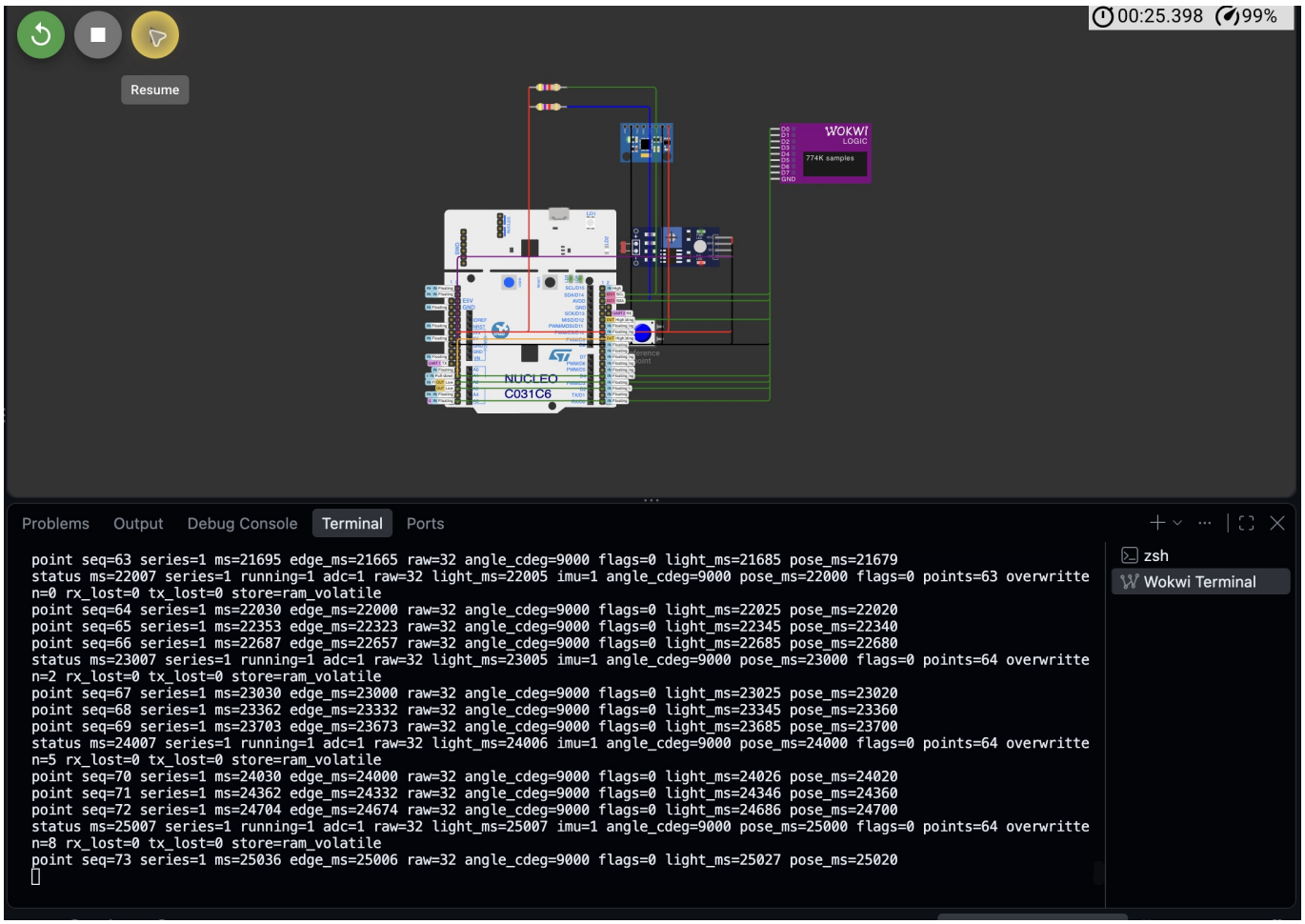


Рисунок 1 – LightBench у Wokwi: схема й UART із точками 63–73

Знімок походить із того самого фінального запуску, що й VCD/UART у репозиторії. На ньому модель призупинена для огляду. Загалом отримано 75 точок за 29.258 с модельного часу; під час зміни входів були паузи. Це не вимір безперервного wall-clock виконання.

Зміна входу LDR дала raw 3413 та 32. Після зміни вектора MPU з (0,0,1)g на (1,0,0)g кут змінився з 0° на 90°. Кнопка створювала один запис за натискання. Детальне декодування I²C, UART і маркерів наведено у ЛР2.

Перевірка середовища:

Перевірка	Фактичний результат
Очищення / збірка	Два чисті запуски, однакові ELF/HEX/BIN
UART	Банер LightBench, selftest, run, status, точки та stop
LED	PA5 відповідає активній серії у записі моделі
Host-тести	Чотири C-набори та три Python-тести на macOS/Linux
Санітизатори	ASan/UBSan без знайдених помилок у цих тестах
Покриття	App+Devices: 100% рядків, 96.92% гілок; без Board/HAL

Журнал проблем:

1. Симптом: firmware не вміщалась у 28 КіБ. Причина: відсутність LTO та зайві бібліотечні перетворення. Виправлення: LTO й обмежене перетворення норми; фінальний розмір 23936 байтів.
2. Симптом: перетворення int64 на float підтягувало double helpers. Причина: програмна арифметика M0+. Виправлення: після перевірки меж квадрат норми перетворюється через uint32.
3. Симптом: LTO давав попередження про assembler listing. Причина: per-file listing не відповідав LTO. Виправлення: прибрано прапорці listing, фінальні журнали без warning.
4. Симптом: банер мав build=local. Причина: контейнер не містить git. Виправлення: кореневий Makefile передає parent commit і checksum.

5. Симптом: у таблиці роз’ємів PB0/PB1 мали неправильні Arduino-позначення. Причина: змішано MCU pin і номер роз’єму. Виправлення: PB0=D10, PB1=A3 за UM2953, рис. 7, табл. 11; перевірено GPIO схеми.

Зміни збірки та ідентифікації зафіксовано у коміті `6469e71`, остаточну таблицю pinout наведено в підсумковій документації. Журнал описує перевірені виправлення, а не порядок лабораторних за датами комітів.

Висновки:

Підготовлено середовище LightBench на Ubuntu 24.04, Docker, STM32 HAL і Wokwi. Дві чисті збірки дали однакові бінарні результати. UART і LED перевірено у поточній інтегрованій прошивці. Визначено 14 функціональних вимог, правила часу та три компоненти з різними інтерфейсами.

Світло вимірюється некаліброваним raw, кут придатний лише для близького до статичного вектора. У ЛР2 реалізовано часові потоки й RAM-журнал. Постійне відновлення після reset, профайлінг ЛР3 та фізична точність тут не підтверджені.

Використані джерела:

1. Код LightBench, PRD, pinout і результати: https://github.com/Th1sMaxOn/LightBench. Поточна версія перевірена локально.
2. STMicroelectronics, DS13867, STM32C031x4/x6, Rev. 4, §4: https://www.st.com/resource/en/datasheet/stm32c031c6.pdf.
3. STMicroelectronics, UM2953, Nucleo-64 MB1717, рис. 7, табл. 11: https://www.st.com/resource/en/user_manual/um2953-stm32-nucleo64-board-mb1717-stmicroelectronics.pdf.
4. Wokwi, Nucleo-C031C6 та конфігурація проєкту: https://docs.wokwi.com/parts/board-st-nucleo-c031c6, https://docs.wokwi.com/vscode/project-config.
5. TDK, MPU-6000/6050 Register Map, Rev. 4.2: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map.pdf.